



DriveNow ERP

Car Rental Management Platform

Technical Documentation

DriveNow ERP - Microservices Platform

ITSAR2 - EndTerm Project

Submitted by

Math Auric Ros Badajos

Shirley Ann Salazar

Nathaniel Villalobos

Rea Nicole Yanson

Maika Zambra

BSIT-3B

Submitted to

Joao Roumil Vergara

Course Instructor

Date

June 2026

Table of Contents

Table of Contents	2
1. Executive Technical Summary	3
2. System Architecture Overview	5
2.1 Architectural principles	5
2.2 Tiers	5
3. Architecture Diagrams	7
4. Technology Stack	8
5. Microservices Breakdown	9
6. Service Responsibilities	10
6.1 auth-service - Operations Hub (:8001)	10
6.2 fleet-service - Fleet (:8002)	10
6.3 rental-service - Rentals (:8003)	10
6.4 crm-service - Customers (:8004)	10
6.5 billing-service - Billing (:8005)	10
6.6 maintenance-service - Maintenance (:8006)	10
6.7 analytics-service - Analytics (:8007)	10
6.8 client-service - Client Portal (:8008)	10
7. Inter-Service Communication	12
7.1 Reliability characteristics	12
8. API Documentation	13
8.1 Internal API endpoints	13
8.2 Conventions	13
9. Authentication & Authorization	15
9.1 End-user authentication (Fortify)	15
9.2 Cross-service single sign-on (stateless JWT)	15
9.3 Internal API authorization (service token)	15
9.4 Role-based access control	15
10. Database Design and ERD	17
10.1 auth-service - users	17
10.2 fleet-service - vehicles	18
10.3 rental-service - rentals	18

10.4 crm-service - customers, customer_activities, customer_feedback	18
10.5 billing-service - invoices, payments	19
10.6 maintenance-service - maintenance_records	19
10.7 client-service - bookings, users	19
11. Data Flow Diagrams	20
11.1 Settings broadcast flow	20
12. Module Descriptions	21
13. Frontend Architecture	22
14. Backend Architecture	23
15. Directory Structure	24
16. Configuration Management	25
17. Docker & Deployment Architecture	26
17.1 Image build (multi-stage)	26
17.2 Runtime entrypoint	26
17.3 Orchestration & gateway	26
18. Environment Setup	27
18.1 Principal environment variables	27
18.2 Local bring-up	27
19. Security Implementation	28
20. Error Handling Strategy	29
21. Logging & Monitoring	30
22. Testing Strategy	31
23. CI/CD Process	32
24. Scalability Considerations	33
25. Maintenance Guide	34
25.1 Operational notes	34
26. Troubleshooting Guide	35
27. Future Technical Enhancements	36

1. Executive Technical Summary

DriveNow ERP is a microservices-based Enterprise Resource Planning platform for a single-branch car rental business (DriveNow Car Rental Services, Bacolod City, Philippines). It comprises **eight independently**

deployable services - a required authentication service (the Operations Hub) plus seven functional modules - fronted by a single **nginx API gateway** and backed by **one isolated PostgreSQL database per service**. The platform satisfies the capstone brief's mandatory architecture: microservices with dedicated databases, a single entry point, token-based authentication and authorization, inter-service communication, full containerization, and Docker Compose orchestration.

Each service is a **Laravel 13** application rendering a **Vue 3 + TypeScript** single-page interface through **Inertia.js v3**. Cross-service single sign-on is achieved with a stateless, signed **HS256 JSON Web Token** (the `drivenow_token` cookie) - a shared secret *key*, never a shared database. Internal APIs are versioned under `/api/v1` and protected by a shared service token. The entire stack runs under Docker Compose; at the time of writing all nine application containers plus PostgreSQL report healthy.

Scope note. This documentation describes the DriveNow ERP system as it is actually implemented - its data model, services, routes, enumerations, middleware and configuration. Capabilities that are not present (for example, an automated CI/CD pipeline) are explicitly marked **Not Implemented**.

Property	Value
Services	8 (1 auth + 6 admin modules + 1 public portal) + 1 API gateway
Application framework	Laravel 13 (PHP 8.4 runtime)
Frontend	Vue 3 + TypeScript via Inertia.js v3, Tailwind CSS v4, Vite
Database	PostgreSQL 18 - one isolated logical database per service
AuthN / SSO	Laravel Fortify + stateless HS256 JWT cookie (<code>drivenow_token</code>)
AuthZ	Role-based (admin, staff, maintenance, customer) + service-token for internal APIs
Gateway	nginx reverse proxy, host/subdomain routing, single public origin
Orchestration	Docker Compose (multi-stage images, healthchecks)
Current runtime	9 app containers + PostgreSQL - all healthy; endpoints return HTTP 200

2. System Architecture Overview

The platform follows a **service-per-business-capability** decomposition. Every module owns its data, its HTTP API and its user interface, and can be built, deployed and scaled on its own. There is no shared application database; services collaborate exclusively over HTTP.

2.1 Architectural principles

- **Database-per-service.** Eight separate logical databases on one PostgreSQL instance; no service reads another service's tables. This is the brief's one explicit "strictly prohibited" rule and is enforced in code.
- **Single entry point.** An nginx gateway routes by host/subdomain to the owning service, so each service runs at its own root and every server-generated URL stays correct.
- **Stateless identity.** A signed JWT cookie carries identity across services; verification is local to each service (signature + expiry), so no network call or shared session store is required to authenticate a request.
- **Graceful degradation.** Cross-service reads (gateway service classes) fall back to cached or static data and short timeouts, so one service being down does not cascade.
- **Twelve-factor configuration.** All environment-specific values are injected as environment variables and consumed through Laravel's config layer (safe under cached config).

2.2 Tiers

Tier	Components	Responsibility
Presentation	Vue 3 + Inertia SPA per service	Server-driven SPA; pages rendered as Inertia responses
Edge	nginx API gateway (:8080)	Single public origin; subdomain -> service routing; forwards SSO cookie
Application	8 Laravel services (nginx + php-fpm)	Business logic, REST APIs, authorization, rendering
Data	PostgreSQL 18	8 isolated databases; per-service schema and migrations

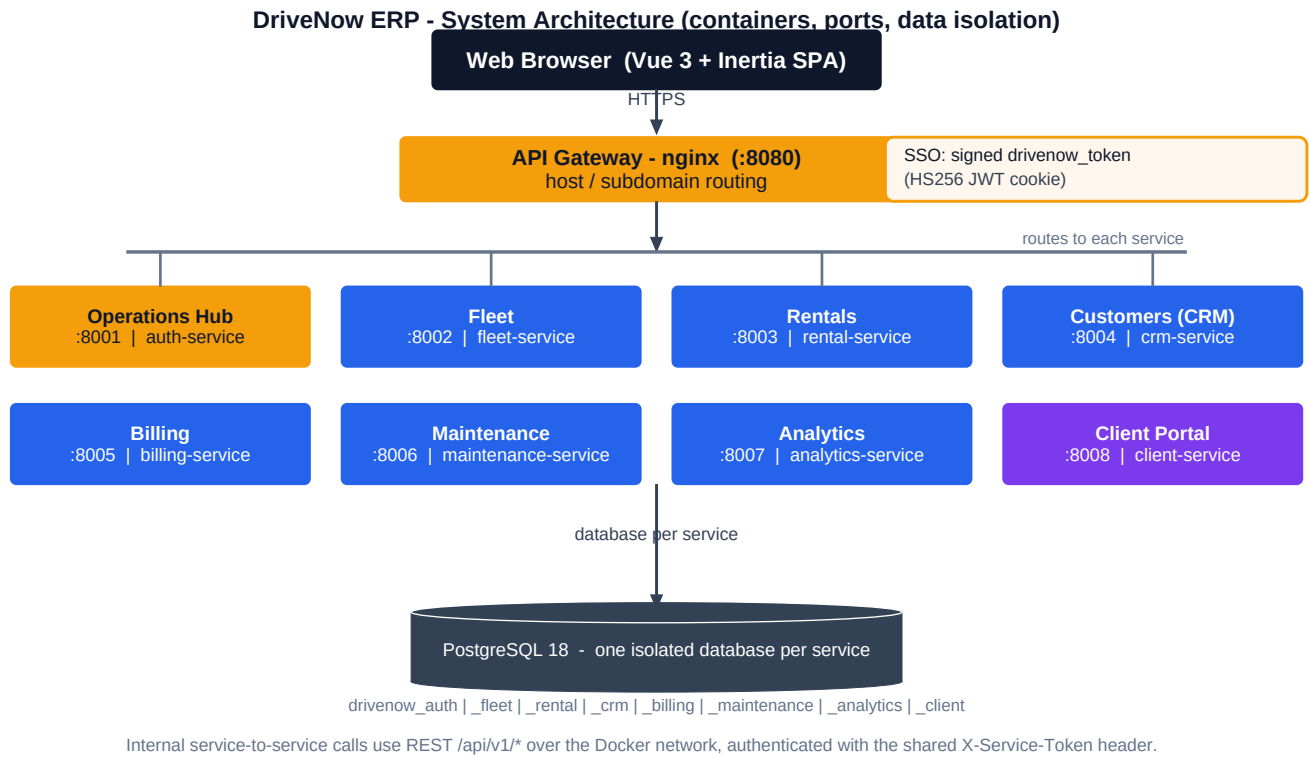


Figure 2-1. System architecture: browser -> gateway -> eight containerized services -> isolated databases.

3. Architecture Diagrams

This section consolidates the principal architecture views. Detailed data and communication views appear in Sections 7, 10 and 17.

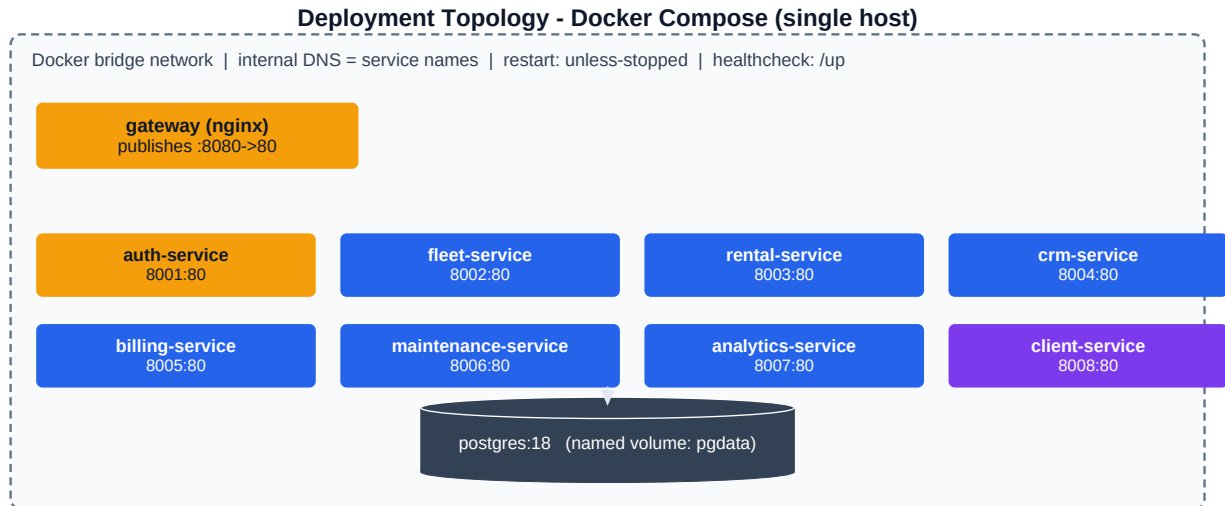


Figure 3-1. Deployment topology under Docker Compose (single host, internal DNS, published ports).

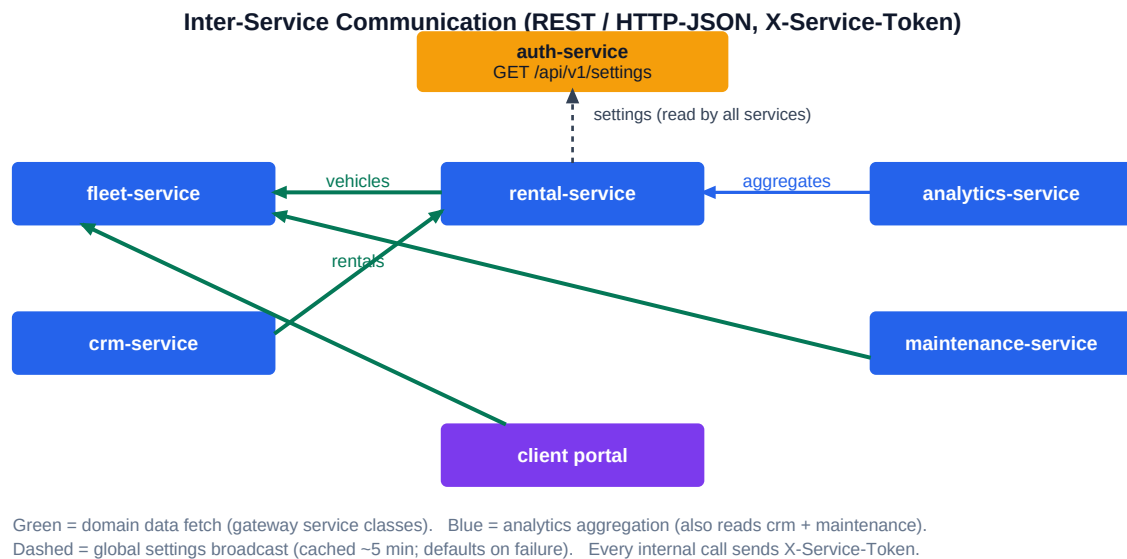


Figure 3-2. Inter-service communication map (REST over the internal network).

4. Technology Stack

The stack is deliberately homogeneous (a single language per tier) to keep eight services maintainable by a small team while still honouring strict service isolation. Each choice is justified against the brief's requirements.

Layer	Technology	Role / Justification
Language (backend)	PHP 8.4	Runtime for all services (Docker uses php:8.4-fpm)
Framework	Laravel 13	Mature batteries-included framework; routing, ORM, validation, queues
Auth scaffolding	Laravel Fortify	Headless login, registration, password reset, email verify, 2FA (TOTP), passkeys
SPA bridge	Inertia.js v3	Server-driven SPA - no separate API client needed for the UI
Typed routes	Laravel Wayfinder	Generates typed TS route/action helpers from PHP routes at build time
Frontend	Vue 3 + TypeScript	Reactive component UI with static typing
UI / styling	Tailwind CSS v4, reka-ui, lucide-vue-next, vue-sonner	Utility CSS, headless components, icons, toasts
Build tool	Vite	Asset bundling; integrated with Wayfinder during image build
Database	PostgreSQL 18	Reliable RDBMS; one database per service
Testing	Pest 4	PHP testing framework (feature/unit/browser capability)
Web server	nginx + php-fpm	Per-service runtime; gateway is also nginx
Process mgr	supervisord	Runs php-fpm and nginx inside each container
Containerization	Docker, Docker Compose	Per-service multi-stage images; single-file orchestration
Code style	Laravel Pint	PSR-12 formatting

Why REST over the alternatives. The brief permits REST, gRPC or a message broker. DriveNow uses **REST/HTTP-JSON** because the interactions are simple synchronous reads (e.g., "list available vehicles"), Laravel's HTTP client makes them first-class, and they require no extra broker infrastructure - keeping the deployment within the Docker Compose minimum while remaining easy to defend and reason about.

5. Microservices Breakdown

The platform implements **seven functional modules in addition to the required authentication service**, exceeding the brief's minimum of four. One module (Client Portal) is a public, customer-facing application with its own authentication realm.

#	Service	Port	Database	Primary domain	UI audience
1	auth-service (Operations Hub)	8001	drivenow_auth	Identity, roles, global settings, audit log	Admin / Staff
2	fleet-service	8002	drivenow_fleet	Vehicle inventory & status	Admin / Staff
3	rental-service	8003	drivenow_rental	Reservations & rental lifecycle	Admin / Staff
4	crm-service	8004	drivenow_crm	Customers, loyalty, feedback	Admin / Staff
5	billing-service	8005	drivenow_billing	Invoices & payments	Admin / Staff
6	maintenance-service	8006	drivenow_maintenance	Vehicle maintenance records	Admin / Maintenance
7	analytics-service	8007	drivenow_analytics	Reporting & BI dashboards	Admin / Staff
8	client-service (Client Portal)	8008	drivenow_client	Public browsing & self-service bookings	Customers (public)
-	gateway	8080	-	Single entry point (reverse proxy)	-

Note on the Analytics database. drivenow_analytics contains no domain tables. Analytics composes its figures on demand by reading sibling services' /api/v1 endpoints (concurrently) and, when a peer is unreachable, falls back to representative values and flags the dashboard as showing degraded data.

6. Service Responsibilities

6.1 auth-service - Operations Hub (:8001)

System of record for identity and global configuration. Provides Fortify-based authentication (login, registration, password reset, email verification, two-factor authentication and passkeys), issues the cross-service JWT on login, manages staff users and roles, owns the global settings (business name, branding, currency, timezone) consumed by every service, and records a system `activity_logs` feed. Exposes GET `/api/v1/settings`.

6.2 fleet-service - Fleet (:8002)

Manages the vehicle inventory: make, model, year, plate, category, status, branch, daily rate, seats, transmission, fuel type, mileage and photo. Vehicle status drives availability across the platform. Exposes GET `/api/v1/vehicles` and `/vehicles/{id}`.

6.3 rental-service - Rentals (:8003)

Owns the reservation lifecycle (pending -> approved -> active -> completed, plus cancelled). Fetches available vehicles from fleet, performs date-overlap conflict checks, computes totals, and stores a *snapshot* of customer and vehicle details so a rental record is self-contained. Exposes GET `/api/v1/rentals` and `/rentals/{id}`.

6.4 crm-service - Customers (:8004)

Maintains customer profiles, loyalty tier (regular/silver/gold/platinum) and points, status (active/inactive/blacklisted), an activity timeline and feedback ratings. Reads a customer's rental history from rental-service. Exposes GET `/api/v1/customers` and `/customers/{id}`.

6.5 billing-service - Billing (:8005)

Generates invoices (subtotal, penalty, discount, total, balance), records payments by method (cash/GCash/card) within a database transaction, recalculates invoice status (unpaid/partial/paid/overdue/refunded) from the balance and due date, and supports refunds. Exposes GET `/api/v1/invoices` and `/invoices/{id}`.

6.6 maintenance-service - Maintenance (:8006)

Tracks maintenance records by type (inspection/repair/scheduled/damage), status (scheduled -> in progress -> completed, plus cancelled) and severity (low/medium/high), with cost and odometer. Reads vehicle references from fleet. Exposes GET `/api/v1/maintenance` and `/maintenance/{record}`.

6.7 analytics-service - Analytics (:8007)

Read-only business intelligence. Aggregates KPIs (revenue, fleet utilization, rental trends, payment mix, customer retention) by concurrently calling sibling APIs, and renders dashboards and a date-range report with client-side CSV export. Exposes GET `/api/v1/summary`.

6.8 client-service - Client Portal (:8008)

The public customer website with its **own authentication realm** (separate user base in `drivenow_client`). Customers browse the catalogue (vehicles fetched from fleet), view details and create self-service bookings,

and manage their profile (including avatar). It is the gateway's default origin.

7. Inter-Service Communication

Services communicate **synchronously over REST** using Laravel's HTTP client, encapsulated in dedicated *gateway* service classes (for example VehicleGateway, RentalGateway, SettingsService). The brief requires at least two services to communicate; DriveNow has several such integrations.

Caller	Callee	Endpoint	Purpose
rental-service	fleet-service	GET /api/v1/vehicles?status=available	Vehicle picker for new reservations
maintenance-service	fleet-service	GET /api/v1/vehicles	Vehicle selector for maintenance records
client-service	fleet-service	GET /api/v1/vehicles	Public catalogue & booking
crm-service	rental-service	GET /api/v1/rentals	Customer rental history
analytics-service	fleet/rental/crm/maintenance	GET /api/v1/* (concurrent)	KPI aggregation
all admin + client	auth-service	GET /api/v1/settings	Global branding/currency/timezone

7.1 Reliability characteristics

- **Short timeouts** (~2 s) on every cross-service call to avoid head-of-line blocking.
- **Fallbacks**: fleet/rental gateways return a cached/static catalogue or empty set on failure; the settings client returns sensible defaults; analytics flags degraded data.
- **Concurrency**: analytics issues its four fetches with `Http::pool()` to cap cold latency.
- **Caching**: settings (~5 min) and some catalogues (~2 min) are cached to reduce chatter.
- **Authentication**: every internal call sends the shared X-Service-Token header (Section 9).

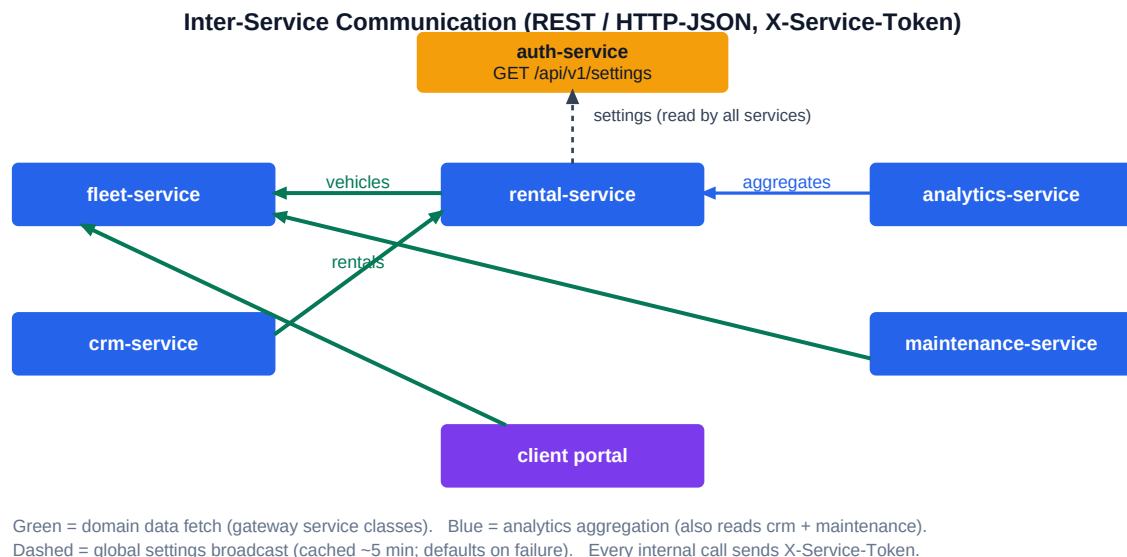


Figure 7-1. Inter-service communication, transport and degradation behaviour.

8. API Documentation

Every service exposes a small, **versioned, read-oriented internal API** under `/api/v1`, guarded by the `VerifyServiceToken` middleware. These endpoints are for server-to-server consumption (they return JSON resources); the interactive user features are delivered through Inertia web routes (Section 14). Mutations are performed through the authenticated web routes, not the internal API.

8.1 Internal API endpoints

Service	Method	Path	Returns	Auth
auth	GET	<code>/api/v1/settings</code>	Public settings payload	X-Service-Token
fleet	GET	<code>/api/v1/vehicles</code>	Vehicle collection (filterable)	X-Service-Token
fleet	GET	<code>/api/v1/vehicles/{vehicle}</code>	Single vehicle	X-Service-Token
rental	GET	<code>/api/v1/rentals</code>	Rental collection	X-Service-Token
rental	GET	<code>/api/v1/rentals/{rental}</code>	Single rental	X-Service-Token
crm	GET	<code>/api/v1/customers</code>	Customer collection	X-Service-Token
crm	GET	<code>/api/v1/customers/{customer}</code>	Single customer	X-Service-Token
billing	GET	<code>/api/v1/invoices</code>	Invoice collection	X-Service-Token
billing	GET	<code>/api/v1/invoices/{invoice}</code>	Single invoice	X-Service-Token
maintenance	GET	<code>/api/v1/maintenance</code>	Maintenance collection	X-Service-Token
maintenance	GET	<code>/api/v1/maintenance/{record}</code>	Single record	X-Service-Token
analytics	GET	<code>/api/v1/summary</code>	Aggregated KPI summary	X-Service-Token

8.2 Conventions

- **Versioning:** all internal routes are prefixed `/api/v1` with route names `api.v1.*`, allowing future versions to coexist.
- **Auth:** a missing or wrong X-Service-Token yields 401 (compared with `hash_equals`).
- **Format:** JSON; collections are returned under a `data` key (Laravel API Resources).
- **Health:** every service additionally exposes `GET /up` (Laravel health endpoint) used by Docker healthchecks.

8.3 Example request

Example	
Request	<code>curl -H 'X-Service-Token: <shared-token>' \</code> <code>http://fleet-service/api/v1/vehicles?status=available</code>
Response	<code>200 OK { "data": [{ "id": 1, "name": "...", "plate_number": "...",</code> <code>"category_label": "Sedan", "daily_rate": 60.0, "status": "available", ... }]</code> <code>}</code>
Failure	<code>401 Unauthorized - "Invalid or missing service token."</code>

The complete machine-readable API specification is provided as **docs/openapi.yaml** (OpenAPI 3.0.3). It documents all 12 internal endpoints across the 7 services, including request parameters, full response schemas, enum values, and worked examples - derived directly from the resource classes and route definitions. Load the file in Swagger UI (<https://editor.swagger.io>) or Redoc for an interactive reference.

9. Authentication & Authorization

9.1 End-user authentication (Fortify)

auth-service and client-service use **Laravel Fortify**. Enabled features (from config/fortify.php) are: registration, password reset, email verification, two-factor authentication (TOTP) and passkeys (WebAuthn). auth-service additionally validates that an account's status is active at login, so a deactivated staff member cannot sign in.

9.2 Cross-service single sign-on (stateless JWT)

On successful login, auth-service issues a signed **HS256 JWT** via `JwtService::issue()` containing the claims `sub`, `name`, `email`, `role`, `role_label`, `iat`, `exp` (2-hour TTL) and sets it as the `drivenow_token` cookie. The cookie is intentionally left unencrypted by Laravel (listed in the `encryptCookies` exceptions) so every sibling can read the raw token. Each service's `AuthenticateWithJwt` middleware verifies the signature and expiry locally with the shared `JWT_SECRET` and, on success, sets the request user statelessly - no service touches another service's database or a shared session store.

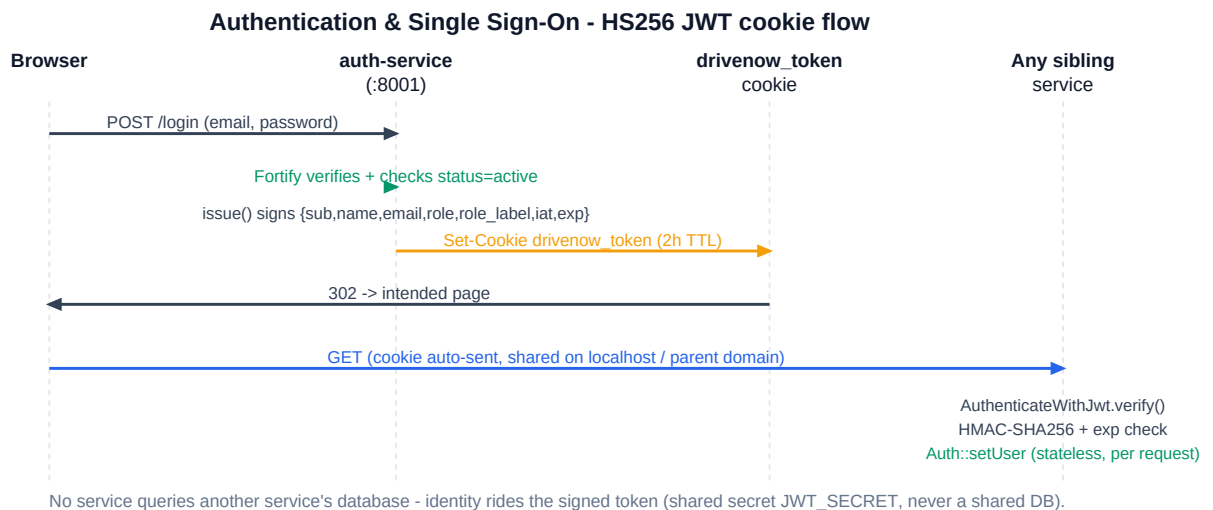


Figure 9-1. Login issues a signed JWT cookie; each service verifies it locally (shared key, not a shared DB).

9.3 Internal API authorization (service token)

Server-to-server endpoints are protected by `VerifyServiceToken`, which requires the shared `X-Service-Token` header (constant-time compared) and returns 401 otherwise. This token is distinct from the JWT and never leaves the server side.

9.4 Role-based access control

Four roles exist: **admin**, **staff**, **maintenance**, **customer**. Interactive routes are wrapped in the `auth` middleware, and **mutating routes are additionally gated by role** via the `EnsureUserHasRole` middleware (alias `role`). Read/dashboard routes remain accessible to any authenticated staff identity.

Service	Mutations gated to	Read access
fleet	role:admin,staff	authenticated

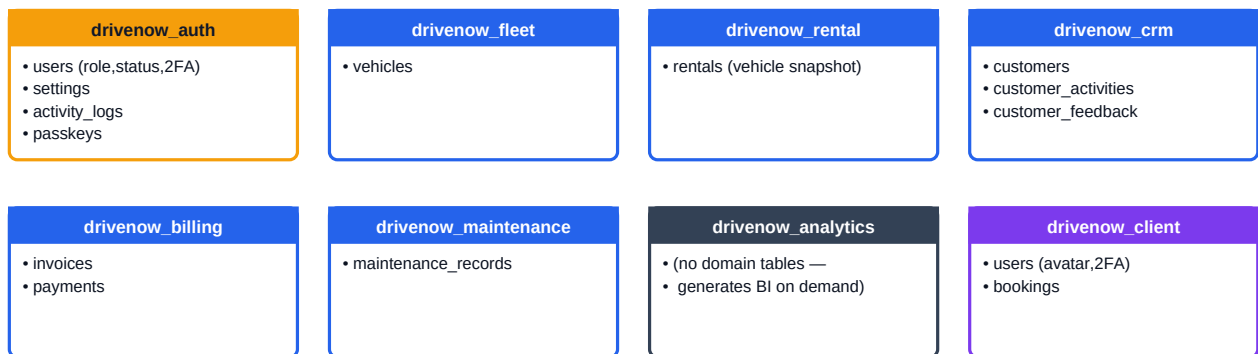
Service	Mutations gated to	Read access
rental	role:admin,staff	authenticated
crm	role:admin,staff	authenticated
billing	role:admin,staff	authenticated
maintenance	role:admin,maintenance	authenticated
analytics	(read-only - no mutations)	authenticated
auth (system & users settings)	role:admin	authenticated

Verified behaviour: a customer-role token attempting a fleet mutation receives **HTTP 403**; an admin token is permitted. Guests are redirected to the Hub login.

10. Database Design and ERD

Each service owns a dedicated PostgreSQL database and migrations. There are **no foreign keys across services**; cross-service references are stored as value snapshots to preserve isolation. The following data dictionary documents the database schema of each service.

Logical Data Architecture - database-per-service (no cross-service foreign keys)



Within-service relationships (physical FKs):

- crm: customers 1-* customer_activities, customers 1-* customer_feedback
- billing: invoices 1-* payments
- auth / client: users 1-* passkeys
- client: users 1-* bookings

Cross-service references are value snapshots, not FKs: rentals store vehicle_id + vehicle_name/plate; invoices store rental_id + rental_reference; maintenance stores vehicle_id + vehicle_name/plate; bookings store vehicle_id + vehicle_name. This preserves isolation (a service keeps working if a peer is offline) at the cost of eventual consistency - an intentional microservices trade-off.

Figure 10-1. Logical data architecture - databases, principal entities and in-service relationships.

10.1 auth-service - users

Column	Type	Notes
id	bigint	PK
name	varchar	
email	varchar	unique login
email_verified_at	timestamp	nullable
password	varchar	hashed
role	varchar	admin staff maintenance customer
phone	varchar	nullable
status	varchar	active inactive (enforced at login)
two_factor_secret / _recovery_codes / _confirmed_at	text/timestamp	2FA (TOTP)
remember_token	varchar	Laravel remember-me

Other auth tables: settings (key, value, group, type), activity_logs (event, description, level, causer), passkeys (user_id, name, credential_id, credential).

10.2 fleet-service - vehicles

Column	Type	Notes
id	bigint	PK
make / model	varchar	
year	smallint	
plate_number	varchar	unique
category	varchar	sedan hatchback suv van pickup
status	varchar	available reserved rented maintenance inactive
branch	varchar	
daily_rate	numeric	
seats	smallint	
transmission / fuel_type	varchar	
color	varchar	nullable
mileage	integer	
image_url	varchar	nullable
notes	text	nullable

10.3 rental-service - rentals

Column	Type	Notes
id	bigint	PK
reference	varchar	RNT-XXXXXXX, unique
customer_name / _email / _phone	varchar	snapshot
vehicle_id	bigint	logical ref to fleet (no FK)
vehicle_name / vehicle_plate	varchar	snapshot
pickup_branch	varchar	
pickup_date / return_date	date	
days	smallint	
daily_rate / subtotal / total	numeric	
status	varchar	pending approved active completed cancelled
approved_at / released_at / returned_at / cancelled_at	timestamp	workflow timestamps

10.4 crm-service - customers, customer_activities, customer_feedback

Table	Key columns
customers	name, email, phone, address, city, status (active inactive blacklisted), tier (regular silver gold platinum), loyalty_points, total_rentals, total_spent, blacklist_reason, joined_at
customer_activities	customer_id (FK), type, description
customer_feedback	customer_id (FK), rating (smallint), comment

10.5 billing-service - invoices, payments

Table	Key columns
invoices	invoice_number, customer_name/_email, rental_id + rental_reference (snapshot), subtotal, penalty, discount, total, amount_paid, balance, status (unpaid partial paid overdue refunded), due_date, issued_at
payments	invoice_id (FK), method (cash gcash card), amount, reference, paid_at

10.6 maintenance-service - maintenance_records

Column group	Columns
Identity	reference, vehicle_id (logical ref), vehicle_name, vehicle_plate
Classification	type (inspection repair scheduled damage), status (scheduled in_progress completed cancelled), severity (low medium high)
Detail	title, description, cost, odometer, scheduled_date, completed_date, notes

10.7 client-service - bookings, users

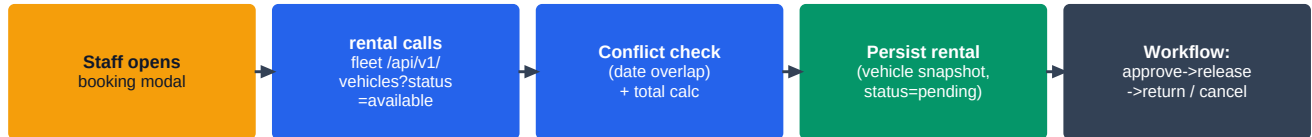
Table	Key columns
bookings	user_id (FK), reference (BK-XXXXXXXX), vehicle_id (logical ref), vehicle_name/_category/_plate (snapshot), daily_rate, pickup_date, return_date, days, total, status
users	name, email, password, avatar, two_factor_* (own auth realm)

analytics-service has no domain tables - only the framework user's table exists. Its figures are derived at request time from sibling APIs.

11. Data Flow Diagrams

The most representative cross-service flow is creating a reservation, which touches fleet (availability), rental (persistence and workflow) and, downstream, billing and analytics.

Representative Data Flow - staff creates a reservation (rental-service)



Billing later issues an invoice referencing the rental (rental_id + rental_reference); analytics reads rentals for KPIs.
If fleet is unreachable the modal falls back to a static catalogue so the booking UI keeps working (graceful degradation).

Figure 11-1. Reservation creation and downstream effects.

11.1 Settings broadcast flow

On any page load, each service's SettingsService fetches GET /api/v1/settings from auth-service (cached ~5 min) and shares the result as an Inertia prop, so branding, currency and timezone are consistent platform-wide without duplicating configuration. If auth-service is unreachable the service falls back to built-in defaults.

12. Module Descriptions

Each functional module follows the same internal layering (Section 14) and encodes its business states as PHP backed enums, which are the single source of truth for statuses, categories and types.

Module	Enum	Values
auth	UserRole	admin, staff, maintenance, customer
fleet	VehicleStatus	available, reserved, rented, maintenance, inactive
fleet	VehicleCategory	sedan, hatchback, suv, van, pickup
rental	RentalStatus	pending, approved, active, completed, cancelled
crm	CustomerStatus	active, inactive, blacklisted
crm	CustomerTier	regular, silver, gold, platinum
billing	InvoiceStatus	unpaid, partial, paid, overdue, refunded
billing	PaymentMethod	cash, gcash, card
maintenance	MaintenanceStatus	scheduled, in_progress, completed, cancelled
maintenance	MaintenanceType	inspection, repair, scheduled, damage
maintenance	MaintenanceSeverity	low, medium, high

Loyalty tier in CRM is derived from accumulated points; invoice status in billing is recalculated from the outstanding balance and due date; rental and maintenance expose explicit workflow transitions that stamp the corresponding timestamp columns.

13. Frontend Architecture

The frontend is a **server-driven SPA**. Laravel controllers return Inertia responses naming a Vue page component and its props; Inertia performs client-side navigation without a hand-written API client. This keeps authorization and validation on the server while delivering an app-like experience.

- **Pages & layouts:** Vue 3 SFCs under `resources/js/pages`; a shared `AppLayout` with the custom navy `ErpSidebar` for admin services; a `PublicLayout` for the client portal.
- **Typed routing:** Laravel Wayfinder generates TypeScript helpers from PHP routes (imported from `@/routes / @/actions`), so the UI calls backend routes type-safely.
- **Shared props:** a `HandleInertiaRequests` middleware shares `auth.user`, global settings, flash toasts and the config-driven `serviceUrls` map (cross-service navigation).
- **Design system:** Tailwind CSS v4 with a navy + amber theme, reka-ui headless components, lucide icons, vue-sonner toasts, dark/light mode, and currency/number formatting driven by the global settings.

Cross-service links are config-driven. Sidebar and dashboard links to sibling services are injected via the `serviceUrls` prop (from `config('services.public')`), overridable per environment with `*_PUBLIC_URL` rather than hard-coded - so the UI works behind the gateway or a public host without code changes.

14. Backend Architecture

Each service is a standard layered Laravel application:

Layer	Responsibility	Examples
Routing	HTTP entry points	routes/web.php (Inertia), routes/api.php (/api/v1)
Middleware	Cross-cutting concerns	AuthenticateWithJwt, VerifyServiceToken, EnsureUserHasRole, RecordActivity, HandleInertiaRequests, trustProxies
Controllers	Request handling & responses	DashboardController, VehicleController, RentalController, ...
Form Requests	Validation & authorization	StoreRentalRequest, StoreVehicleRequest, ...
Domain	Business rules & state	Eloquent models + backed Enums + Repositories
Services / Gateways	Cross-service & shared logic	VehicleGateway, RentalGateway, SettingsService, JwtService, AnalyticsService
Resources	API serialization	VehicleResource, RentalResource, ...
Persistence	Data access	Eloquent ORM over PostgreSQL; per-service migrations, factories, seeders

Workflow-style operations (approve/release/return/cancel a rental; start/complete/cancel a maintenance record; record a payment/refund) are explicit controller actions bound to POST routes and protected by the role middleware.

15. Directory Structure

All eight services share the same conventional Laravel + Inertia layout. A representative service tree (abbreviated):

Path	Contents
app/Enums/	Backed enums (status, category, type, role)
app/Http/Controllers/	Web + Api/ controllers
app/Http/Middleware/	AuthenticateWithJwt, VerifyServiceToken, EnsureUserHasRole, RecordActivity, HandleInertiaRequests
app/Http/Requests/	Form Request validation classes
app/Http/Resources/	API resource transformers
app/Models/	Eloquent models
app/Services/	JwtService, *Gateway, SettingsService, domain services
app/Providers/	AppServiceProvider (forceRootUrl, defaults), FortifyServiceProvider (auth only)
bootstrap/app.php	Middleware wiring, routing, role alias, trustProxies
config/	app.php, services.php (jwt_secret, service_token, internal, public), fortify.php
database/	migrations/, factories/, seeders/
resources/js/	pages/, layouts/, components/ErpSidebar.vue, lib/ (format, flashToast), composables/
routes/	web.php, api.php, console.php
docker/	nginx.conf, php.ini, supervisord.conf, entrypoint.sh
Dockerfile	.dockerignore - multi-stage build

Repository root additionally contains `docker-compose.yml`, the `gateway/` directory (nginx config + Dockerfile), `docker/postgres/init-databases.sh`, secret-generation scripts under `scripts/`, and the project documentation (`DEPLOYMENT.md`, `audit.md`, this `docs/` folder).

16. Configuration Management

Configuration follows twelve-factor principles. Environment variables are injected by Docker Compose and read exclusively through Laravel's `config()` layer (never `env()` in application code), which is safe because the container caches config at boot. Key configuration in `config/services.php`:

Config key	Source	Purpose
<code>services.jwt_secret</code>	<code>JWT_SECRET</code>	Shared HS256 signing key for SSO
<code>services.service_token</code>	<code>SERVICE_TOKEN</code>	Shared internal API token
<code>services.internal.*</code>	<code>*_SERVICE_URL</code>	In-cluster DNS URLs for server-to-server calls
<code>services.public.*</code>	<code>*_PUBLIC_URL</code>	Browser-facing URLs for cross-service navigation
<code>services.auth.url</code>	<code>AUTH_SERVICE_URL</code>	Auth origin (settings, guest-redirect)

Global *business* configuration (business name, logo, currency symbol, timezone) is not stored in files but in auth-service's `settings` table and broadcast to all services at runtime, so an administrator can rebrand the whole platform from the Operations Hub.

Internal vs public URLs. Server-to-server calls use Docker DNS names (`http://fleet-service`) because `localhost:800X` inside a container refers to the container itself; browser links use the published `*_PUBLIC_URL`. Keeping these distinct is essential and is encoded in config.

17. Docker & Deployment Architecture

17.1 Image build (multi-stage)

Each service uses a two-stage Dockerfile. **Stage 1** (php:8.4-cli + Composer + Node 22) installs PHP dependencies and builds the Vite/Inertia assets - Laravel Wayfinder runs during the Vite build to regenerate typed route helpers. **Stage 2** (php:8.4-fpm + nginx + supervisord) is the slim runtime that serves public/ and proxies PHP to php-fpm.

17.2 Runtime entrypoint

1. Ensure writable storage and create the public storage symlink.
2. Wait for PostgreSQL, then run `migrate --force` (idempotent).
3. Optionally seed demo data on first boot when `APP_AUTO_SEED=true`.
4. Warm config, route and view caches (after env is present).
5. Hand off to supervisord, which runs php-fpm and nginx.

17.3 Orchestration & gateway

`docker-compose.yml` defines all eight services, PostgreSQL and the gateway on one bridge network, sharing an environment anchor (database, secrets, service URLs) and per-service `APP_URL`/ports. `docker/postgres/init-databases.sh` creates the eight databases on first boot. Each service publishes `800X:80`; the gateway publishes `8080:80` and routes by subdomain (`auth.*`, `fleet.*`, ..., `default` -> `client portal`). Healthchecks `poll /up`.

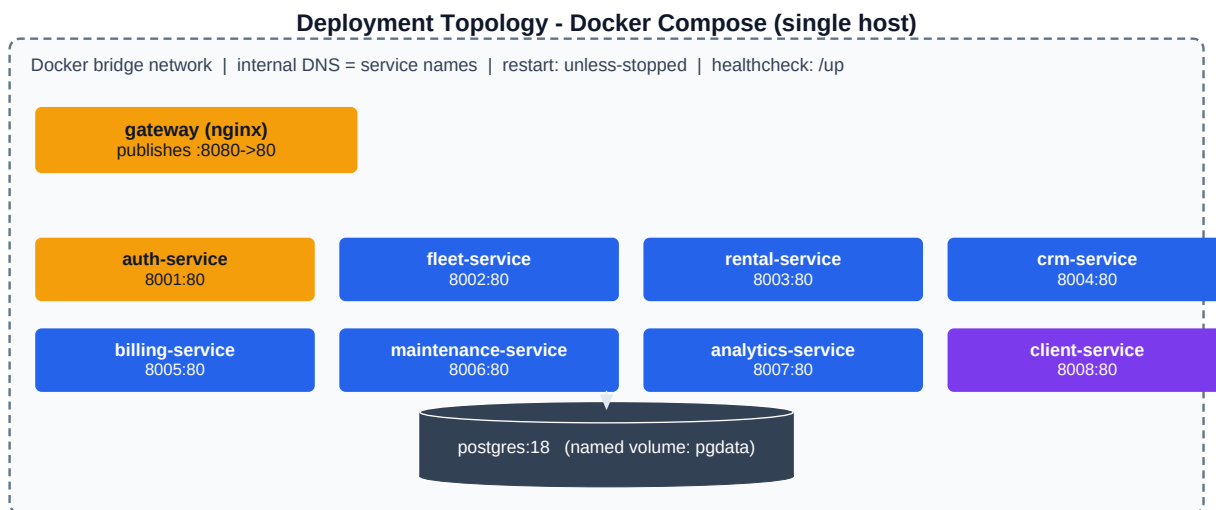


Figure 17-1. Container, network and storage topology.

18. Environment Setup

18.1 Principal environment variables

Variable	Example / default	Purpose
APP_ENV / APP_DEBUG	production / false	Runtime mode
APP_URL	http://localhost:8001	Per-service public base URL
ASSET_URL	http://localhost:8001	Asset base (now wired via app.asset_url)
DB_CONNECTION / DB_HOST	pgsql / postgres	Database connection
DB_DATABASE	drivenow_	Per-service database name
JWT_SECRET	(shared secret)	SSO token signing - identical across services
SERVICE_TOKEN	(shared secret)	Internal API token - identical across services
*_SERVICE_URL	http://fleet-service	Internal DNS for server-to-server calls
*_PUBLIC_URL	http://localhost:8002	Browser-facing sibling URLs
SESSION_DOMAIN	(empty) / .example.com	Cross-subdomain SSO cookie scope
APP_AUTO_SEED	false / true (first boot)	Seed demo data on initial deploy

18.2 Local bring-up

1. Generate secrets: `scripts/generate-secrets.ps1` (or `.sh`) writes `.env`.
2. Set `APP_AUTO_SEED=true` for the very first run.
3. `docker compose up -d --build`.
4. Access services on `http://localhost:8001-8008`, or the gateway on `:8080`.

Important. Application source is baked into the image (`COPY . .`), so code changes require `docker compose up -d --build` - a plain restart runs stale code.

19. Security Implementation

- **Token-based auth** (HS256 JWT) with signature + expiry verification; the SSO cookie is `httpOnly` and `SameSite=Lax`.
- **Service-to-service authentication** via a shared header token, constant-time compared; internal APIs return 401 without it.
- **Role-based authorization** on every mutating endpoint (verified customer->403, admin->allowed).
- **Account status enforcement** at login (inactive users blocked) and a last-admin guard preventing lockout of the final administrator.
- **Password hashing** (bcrypt) and optional **2FA (TOTP)** and **passkeys** via Fortify.
- **CSRF protection** on web routes; **cookie encryption** for all cookies except the explicitly exempt SSO/appearance cookies.
- **Trusted proxies** configured so client IP and scheme are correct behind the gateway (accurate rate-limiting and audit IPs).
- **Validation** via Form Requests; file uploads constrained by type/size with randomized names.
- **Production hardening**: Laravel enforces stronger password rules and prohibits destructive database commands when `APP_ENV=production`.

Known residual risks (disclosed for the defense). (1) Because SSO is stateless, a deactivated user's existing JWT remains valid until it expires (≤ 2 h); a token denylist would close this. (2) The shared `JWT_SECRET/SERVICE_TOKEN` must be kept secret and rotated carefully. These are documented trade-offs, not defects.

20. Error Handling Strategy

- **Validation errors** are returned as Laravel validation responses and surfaced inline in the Inertia forms.
- **Authorization failures** produce 403; unauthenticated web requests redirect to the Hub login; missing service tokens produce 401.
- **Cross-service failures** are caught in the gateway service classes (try/catch + short timeout) and degrade to cached/static/empty data instead of failing the page.
- **Offline siblings** on the Operations Hub are shown with an OFFLINE indicator and route to a friendly /service-unavailable page rather than a dead link.
- **API requests** (path `api/*`) receive JSON error responses; the SPA shows toast notifications for flashed success/error messages.
- **Transactional integrity**: payment recording and similar multi-write operations run inside database transactions.

21. Logging & Monitoring

A uniform RecordActivity middleware is appended to the web stack of every service and logs each successful state-changing request (POST/PUT/PATCH/DELETE) to the application log channel with the service name, acting user, route, method, status and client IP. auth-service additionally persists a richer activity_logs feed surfaced on the Hub dashboard.

- **Container logs** stream to stdout/stderr (nginx, php-fpm) and are viewable with `docker compose logs`.
- **Health monitoring**: each service exposes /up; Docker healthchecks poll it; the Hub probes peers live and shows an "X of N services online" indicator.
- **Log channel**: services use the stderr/stack channel in containers for aggregation-friendly output.

Not Implemented. Centralized log aggregation and metrics dashboards (e.g., ELK, Prometheus/Grafana) are not part of the current deployment; logging is structured but per-container.

22. Testing Strategy

The project standardizes on **Pest 4** for PHP testing (feature, unit and browser/smoke testing capability) and uses static and build-time checks as the primary safety net to date.

Check	Tool	What it verifies
Type checking	vue-tsc (npm run types:check)	Frontend TypeScript correctness across all services
PHP syntax	php -l	Server-side syntax of changed files
Code style	Laravel Pint	PSR-12 formatting
Build	Vite (npm run build)	Asset build + Wayfinder generation succeed
DB integrity	migrate:fresh --seed	Schema + seeders apply cleanly
Runtime	route:list, /up, manual HTTP	Routing and live endpoint behaviour

Status of automated tests. Pest is configured in every service. A comprehensive automated feature/browser test suite is **not yet implemented**; verification to date has relied on type-checking, linting, build, migration and manual/HTTP checks (including the role-authorization and gateway checks recorded in `audit.md`). Expanding Pest coverage is a recommended enhancement (Section 27).

23. CI/CD Process

Not Implemented. There is no automated continuous-integration or continuous-deployment pipeline (e.g., GitHub Actions) in the repository at this time. Builds and deployments are performed manually via `docker compose up -d --build`.

The codebase is, however, CI-ready: every quality gate listed in Section 22 is a single command and could be wired into a pipeline. A recommended pipeline would run Pint, type-checking, `php -l/Pest` and a Docker build per service on push, then deploy on a tagged release. This is captured as a future enhancement (Section 27).

24. Scalability Considerations

- **Independent scaling:** because each service is its own container with its own database, hot services (e.g., the public portal or fleet) can be replicated independently behind the gateway.
- **Stateless services:** JWT-based identity means application instances hold no session affinity, so horizontal scaling needs no sticky sessions (set `SESSION_DOMAIN` for cross-subdomain cookies when scaling out).
- **Caching:** settings and catalogues are cached to reduce cross-service and database load.
- **Database:** a single PostgreSQL instance hosts eight databases today; each can be moved to its own instance/host without code change (connection is env-driven), removing the shared bottleneck.
- **Asynchronous work:** the queue connection is configured (database driver) and can absorb future background jobs (e.g., notifications, report generation).

Current deployment targets a single host (Docker Compose), which satisfies the brief. Kubernetes (a bonus in the brief) is **Not Implemented** but the stateless, containerized design is compatible with it.

25. Maintenance Guide

Task	Command / action
Deploy code changes	<code>docker compose up -d --build</code> (NOT a plain restart - source is baked in)
Rebuild one service	<code>docker compose up -d --build</code>
View logs	<code>docker compose logs -f</code>
Run migrations	handled automatically by entrypoint; or <code>docker compose exec php artisan migrate --force</code>
Reseed demo data	set <code>APP_AUTO_SEED=true</code> and <code>docker compose up -d</code> , or run <code>db:seed</code> manually
Health check	<code>curl http://localhost:800X/up</code> (per service) / <code>:8080/healthz</code> (gateway)
Rotate secrets	update <code>JWT_SECRET</code> / <code>SERVICE_TOKEN</code> identically across all services, then rebuild
Add a sibling link	add a <code>*_PUBLIC_URL</code> and a <code>services.public</code> entry; no component edits needed

25.1 Operational notes

- Keep `JWT_SECRET` and `SERVICE_TOKEN` identical across all services or SSO and internal APIs will fail.
- After changing global branding/currency, no redeploy is needed - it propagates from the Hub settings.
- Uploaded files (logos, avatars, vehicle photos) are container-local; mount a shared volume or use S3 for persistence across rebuilds.

26. Troubleshooting Guide

Symptom	Likely cause	Resolution
Login redirects to http://localhost/... (no port) -> browser error	Missing URL::forceRootUrl behind the port-stripping proxy	Ensure AppServiceProvider forces APP_URL as root; rebuild image
All sibling services show OFFLINE on the Hub	Health probe used localhost instead of internal DNS	Probe via services.internal (Docker DNS); already implemented
Internal API returns 401	Missing/incorrect X-Service-Token, or secrets differ between services	Align SERVICE_TOKEN across services; ensure caller sends the header
Cross-service list is empty / shows demo data	Caller used pre-v1 path or omitted token	Call /api/v1/* with X-Service-Token (fixed in gateways)
Code change not taking effect	Image not rebuilt (source baked in)	docker compose up -d --build
npm build fails on lightningcss	Intermittent native optional-dependency issue	Re-run the build; it is non-deterministic
Mutation returns 403 unexpectedly	Role not permitted for that action	Confirm the user's role vs the route's role:* gate
Cross-subdomain SSO not shared	SESSION_DOMAIN unset for a real domain	Set SESSION_DOMAIN=.yourdomain.com

27. Future Technical Enhancements

1. **JWT revocation:** add a short denylist (jti/status) so deactivated accounts lose access immediately rather than at token expiry.
2. **Postman collection:** export a Postman collection from the existing OpenAPI spec (`docs/openapi.yaml`) for easier interactive testing.
3. **Automated tests:** grow Pest feature/browser coverage and wire a CI pipeline (lint, type-check, test, build per service).
4. **Shared package:** extract the duplicated JWT/middleware (`JwtService`, `AuthenticateWithJwt`, `VerifyServiceToken`, `RecordActivity`) into a Composer package so security fixes are made once.
5. **Observability:** centralized log aggregation and metrics/health dashboards.
6. **Persistence for uploads:** shared object storage (S3-compatible) for media.
7. **Kubernetes:** optional manifests/Helm chart for the bonus orchestration target.

This document reflects the **current state** of the system. Future items above are explicitly not yet implemented.